

✓ Fine-tuning a small model

I am going to use LoRA with Unsloth. The idea is to create fine-tune a small model to use in a call center to help route customer queries efficiently and more reliably than relying on a prompt. In particular, we will try to produce a JSON payload that can be used to do the routing. For this experiment I am using only "intent" and "priority" so that we can send it to the right queue and assign higher priority to those requests that are labelled as high.

I leave in my wish list adding other fields in the future like "sentiment" to warn the agent that the customer might be angry and "summary" to help the agent gain information about the issue faster.

Generating synthetic data

I used the following prompt on my local Gemma4:e4b in Ollama to create 200 samples for a call center dataset with labels "intent" and "priority"

```
You are an expert Data Annotator specializing in conversational AI training data. Your sole task is to generate a large, diverse dataset of user prompts for a call center routing system.

GOAL: Generate at least 200 unique and conversational user prompts. The data must be realistic, simulating real chat transcripts.

Definitions and Labels

1. Intents (What is the user trying to do?):
- technical_support: The user is experiencing a functional breakdown or failure (e.g., "My password won't work," "I can't access my account").
- billing_query: The user has questions about money, invoices, payments, or charges (e.g., "Why was I charged twice?", "What is my bill for?").
- product_info: The user is asking for details about a feature, product specifications, or general usage tips (e.g., "How do I use this feature?").

2. Priority (How fast does this need attention?):
- high: The issue is preventing the user from completing a critical, immediate function (e.g., system down, lost access, cannot log in).
- medium: The issue needs resolution but is not time-sensitive (e.g., general questions, requests for policy clarification, non-urgent problems).
- low: The issue is informational, navigational, or merely a casual inquiry (e.g., "What are your business hours?" or "Tell me more about your services").

Data Generation Rules
- Tension between Intent and Priority: The Intent and Priority do not always match. Example: A billing question (intent: billing_query, priority: high).
- Format Consistency: Every single entry must be a JSON object following the specified structure.
- Natural Language: The input text must sound like something a real customer would write.

Desired Output Format
[
  {
    "input_text": "String containing the simulated user query.",
    "intent": "One of the defined intents (e.g., 'technical_support', 'billing_inquiry').",
    "priority": "One of the defined priorities ('high', 'medium', 'low')."
  },
  // ... more objects ...
]

BEGIN YOUR OUTPUT HERE. Please generate a JSON array containing at least 200 distinct and varied examples following all the rules above.
```

✓ Preparing the data for training and testing

```
import json

def pre_data(file_name):
    """
    Prepare training and testing data from a JSON file.
    Args:
        file_name (str): The name of the JSON file containing the data.
    Returns:
        tuple: A tuple containing the training data and testing data.
    """

    training_pc = 0.9 # Percentage of data to be used for training
    print(">>> Preparing data ...")
    with open(file_name, "r") as file:
```

```

        content = file.read()
        file.close()
    data = json.loads(content)
    print(f"Loaded {len(data)} records")

    training_data = data[:int(len(data)*training_pc)]
    print(f"Selecting training data ({int(training_pc*100)}%): {len(training_data)} records")
    testing_data = data[int(len(data)*training_pc):]
    print(f"Selecting testing data ({int((1-training_pc)*100)}%): {len(testing_data)} records")
    return training_data, testing_data

training_data, testing_data = pre_data("call_center.json")
print("\n>>> Inspecting training data ...")
print(json.dumps(training_data[0:2], indent=4))

```

```

>>> Preparing data ...
Loaded 200 records
Selecting training data (90%): 180 records
Selecting testing data (9%): 20 records

>>> Inspecting training data ...
[
  {
    "input_text": "I just can't log into my dashboard, it keeps giving me a 500 error. I need to file my report!",
    "intent": "technical_support",
    "priority": "high"
  },
  {
    "input_text": "How do I adjust the automatic shipment frequency for my subscription package?",
    "intent": "product_info",
    "priority": "medium"
  }
]

```

```
!pip install unsloth trl peft accelerate bitsandbytes
```

```

Requirement already satisfied: cuda-binding==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch<2.11.0,>=2.11.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch<2.11.0,>=2.11.0)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch<2.11.0,>=2.11.0)

```

```
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets!=4.0.*,!=4.1.0,<
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets!=4.0.*,!=4.1.0
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->f
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspe
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->f
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->f
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->f
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=13.8.0
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->datas
```

✓ Checking we are using the GPU

```
import torch
print(f"CUDA available: {torch.cuda.is_available()}")
print(f"GPU: {torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'None'}")
```

```
CUDA available: True
GPU: Tesla T4
```

✓ Get the Unsloth model and tokenizer from HuggingFace

```
from unsloth import FastLanguageModel
import torch

model_name = "unsloth/Phi-3-mini-4k-instruct-bnb-4bit"

max_seq_length = 2048 # Choose sequence length
dtype = None # Auto detection

# Load model and tokenizer
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=model_name,
    max_seq_length=max_seq_length,
    dtype=dtype,
    load_in_4bit=True,
)
```

```
🔥 Unsloth: Will patch your computer to enable 2x faster free finetuning.
🔥 Unsloth Zoo will now patch everything to make training faster!
==((====))== Unsloth 2026.5.8: Fast Mistral patching. Transformers: 5.5.0.
\\  /| Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform: Linux.
O^O/ \_/ \ Torch: 2.10.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.6.0
\  ___/ Bfloat16 = FALSE. FA [Xformers = 0.0.35. FA2 = False]
"-_____" Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!
Loading weights: 100% 291/291 [00:01<00:00, 285.61it/s]
```

✓ Prepare the data for training

```
from datasets import Dataset

def format_prompt(example):
    input = example.pop("input_text", None)
    return f"### Input: {input}\n### Output: {json.dumps(example)}<|endoftext|>"

formatted_data = [format_prompt(item) for item in training_data]
print(f"Formatted data:\n {formatted_data[0:2]}\n")

dataset = Dataset.from_dict({"text": formatted_data})
dataset
```

```
Formatted data:
['### Input: I just can\'t log into my dashboard, it keeps giving me a 500 error. I need to file my report!\n### Output: {"inte

Dataset({
  features: ['text'],
  num_rows: 180
})
```

Add LoRA adapters

```
model = FastLanguageModel.get_peft_model(
    model,
    r=32,  # Set r to 32 to handle the expanded data variety
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj", # Using MLP layers for better reasoning
    ],
    lora_alpha=32,
    lora_dropout=0,  # Optimized for Unsloth
    bias="none",
    use_gradient_checkpointing="unsloth",
    random_state=3407,
)
```

Unsloth 2026.5.8 patched 32 layers with 32 QKV layers, 32 O layers and 32 MLP layers.

Set all the parameters for the training

```
from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    dataset_text_field="text",
    max_seq_length=max_seq_length,
    dataset_num_proc=2,
    args=TrainingArguments(
        per_device_train_batch_size=4,
        gradient_accumulation_steps=2,
        warmup_steps=5,  # ~5% of the total
        num_train_epochs=4,  # 4 epochs to prevent over-memorization
        learning_rate=1e-4,  # Standard, stable rate for small-scale SFT
        fp16=not torch.cuda.is_bf16_supported(),
        bf16=torch.cuda.is_bf16_supported(),
        logging_steps=5,  # Watch the loss change every 5 steps
        optim="adamw_8bit",
        weight_decay=0.01,
        lr_scheduler_type="cosine",  # Smooth decay down to zero
        seed=3407,
        output_dir="outputs",
        save_strategy="no",  # Fast training; save manually at the end instead
        report_to="none",
    ),
)
```

Unsloth: Tokenizing ["text"] (num_proc=6): 100%

180/180 [00:01<00:00, 163.18 examples/s]

Let's get Testing

```
# Train the model
trainer_stats = trainer.train()
```

```

==(====)= Unsloth - 2x faster free finetuning | Num GPUs used = 1
  \ \ / / Num examples = 180 | Num Epochs = 4 | Total steps = 92
0^0/ \_ / \ Batch size per device = 4 | Gradient accumulation steps = 2
 \ \ / / Data Parallel GPUs = 1 | Total batch size (4 x 2 x 1) = 8
  \ \ / / Trainable parameters = 59,768,832 of 3,880,848,384 (1.54% trained)
"-_____"
`use_return_dict` is deprecated! Use `return_dict` instead!
[92/92 01:08, Epoch 4/4]

```

Step	Training Loss
5	2.847996
10	1.827496
15	1.324185
20	1.128832
25	0.913698
30	0.873322
35	0.840299
40	0.880957
45	0.827904
50	0.803823
55	0.729992
60	0.702284
65	0.708962
70	0.696942
75	0.707668
80	0.641622
85	0.629443
90	0.676019

✦ Evaluating the Model with `testing_data`

Now, let's iterate through the `testing_data` to see how well the fine-tuned model predicts the `intent` and `priority` for unseen examples. We will compare the model's output with the ground truth from the dataset.

```

import json

# Counters for correct predictions
correct_intent_predictions = 0
correct_priority_predictions = 0
json_decode_errors = 0

total_samples = len(testing_data)

print(f"\n>>> Evaluating model on {total_samples} test samples...\n")

for i, sample in enumerate(testing_data):
    input_text = sample["input_text"]
    expected_intent = sample["intent"]
    expected_priority = sample["priority"]

    # Prepare messages for the model to generate the JSON output
    # The model was trained with '### Input: {input}\n### Output: {json}' format.
    # We prompt it to complete the '### Output: ' part.
    user_prompt_content = f"### Input: {input_text}\n### Output: "
    messages = [
        {"role": "user", "content": user_prompt_content},
    ]

    inputs = tokenizer.apply_chat_template(
        messages,
        tokenize=True,
        add_generation_prompt=True, # This will add <|assistant|> after user message in the template
        return_tensors="pt",
    ).to("cuda")

```

```

# Generate response
outputs = model.generate(
    input_ids=inputs,
    #max_new_tokens=256,
    use_cache=True,
    temperature=0.1,
    do_sample=True,
    top_p=0.9,
)

response = tokenizer.batch_decode(outputs)[0]

# Extract only the assistant's output (JSON part)
# The actual output looks like this: <|user|>...<|end|><|assistant|> {JSON}<|end|>
assistant_prefix = "<|assistant|> "
end_of_text_suffix = "<|end|>"

extracted_output = ""
start_index = response.find(assistant_prefix)
if start_index != -1:
    start_index += len(assistant_prefix)
    end_index = response.find(end_of_text_suffix, start_index)
    if end_index != -1:
        extracted_output = response[start_index:end_index].strip()

# Parse the extracted JSON
model_intent = None
model_priority = None
try:
    model_output = json.loads(extracted_output)
    model_intent = model_output.get("intent")
    model_priority = model_output.get("priority")
except json.JSONDecodeError:
    json_decode_errors += 1
    print(f"Sample {i+1}: Could not decode JSON from model output.")
    print(f"  Extracted output: '{extracted_output}'")
    print(f"  Full model response: '{response}'")

# Compare and print results
print(f"--- Sample {i+1} ---")
print(f"Input: {input_text}")
print(f"Expected Intent: {expected_intent}, Model Intent: {model_intent}")
print(f"Expected Priority: {expected_priority}, Model Priority: {model_priority}")

if model_intent == expected_intent:
    correct_intent_predictions += 1
    print("Intent: Correct")
else:
    print("Intent: Incorrect")

if model_priority == expected_priority:
    correct_priority_predictions += 1
    print("Priority: Correct")
else:
    print("Priority: Incorrect")
print("\n")

# Print overall accuracy
intent_accuracy = (correct_intent_predictions / total_samples) * 100
priority_accuracy = (correct_priority_predictions / total_samples) * 100

print("\n>>> Evaluation Summary <<<")
print(f"Total Test Samples: {total_samples}")
print(f"Correct Intent Predictions: {correct_intent_predictions}/{total_samples} ({intent_accuracy:.2f}%)")
print(f"Correct Priority Predictions: {correct_priority_predictions}/{total_samples} ({priority_accuracy:.2f}%)")
print(f"JSON Decode Errors: {json_decode_errors}")

```

```

--- Sample 16 ---
Input: I need to talk to a human immediately about the service interruption. This is critical for my business.
Expected Intent: technical_support, Model Intent: technical_support
Expected Priority: high, Model Priority: high
Intent: Correct
Priority: Correct

--- Sample 17 ---
Input: What are your standard payment terms for corporate clients, and are there discounts for annual billing?
Expected Intent: billing_query, Model Intent: billing_query
Expected Priority: medium, Model Priority: medium
Intent: Correct
Priority: Correct

--- Sample 18 ---
Input: Where is the closest authorized dealer that carries this specific model?
Expected Intent: product_info, Model Intent: product_info
Expected Priority: low, Model Priority: low
Intent: Correct
Priority: Correct

--- Sample 19 ---
Input: The application is running incredibly slowly this afternoon. Can someone check the server status?
Expected Intent: technical_support, Model Intent: technical_support
Expected Priority: medium, Model Priority: high
Intent: Correct
Priority: Incorrect

--- Sample 20 ---
Input: If I upgrade next year, will my current base rate protect me from future price hikes?
Expected Intent: billing_query, Model Intent: billing_query
Expected Priority: low, Model Priority: medium
Intent: Correct
Priority: Incorrect

>>> Evaluation Summary <<<
Total Test Samples: 20
Correct Intent Predictions: 18/20 (90.00%)
Correct Priority Predictions: 13/20 (65.00%)
JSON Decode Errors: 0

```

Observations

It is producing JSON well and the intent is high but could be a bit better.

Priority is a bit low:

- All failed predictions are only one level away from the expected. It might be just due to judgement when it is not very clear
- I have noticed that 48% of my sample data is labelled as "medium" ("high" and "low" have more similar numbers), this could bias the prediction, ie, "when I am unsure if say "medium" I will get a better chance to get it right. However, it doesn't seem to be the case. In fact, 6 of the missed predictions happened when the expected priority was "medium"

Another reason could be that the loss ended over 0.5. Let's run another test with more 5 epochs. Since the model is going to see the same 180 examples one extra time, it has a slightly higher risk of overfitting. Let's bump the weight decay to 0.05.

```

from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    dataset_text_field="text",
    max_seq_length=max_seq_length,
    dataset_num_proc=2,
    args=TrainingArguments(
        per_device_train_batch_size=4,
        gradient_accumulation_steps=2,
        warmup_steps=5,
        num_train_epochs=5,          # 5 Epochs
        learning_rate=1e-4,

```

```
fp16=not torch.cuda.is_bf16_supported(),
bf16=torch.cuda.is_bf16_supported(),
logging_steps=5,
optim="adamw_8bit",
weight_decay=0.05, # Higher decay to protect against overfitting
lr_scheduler_type="cosine", # Keeping "cosine" for smooth decay
seed=3407,
output_dir="outputs",
save_strategy="no",
report_to="none"
)
)
```

Unsloth: Tokenizing ["text"] (num_proc=6): 100% 180/180 [00:01<00:00, 164.61 examples/s]

```
trainer_stats = trainer.train()
```

```
==(=====)== Unsloth - 2x faster free finetuning | Num GPUs used = 1
  \ \ / | Num examples = 180 | Num Epochs = 5 | Total steps = 115
0^0/ \ / \ Batch size per device = 4 | Gradient accumulation steps = 2
 \ \ / | Data Parallel GPUs = 1 | Total batch size (4 x 2 x 1) = 8
  "-__-" Trainable parameters = 59,768,832 of 3,880,848,384 (1.54% trained)
`use_return_dict` is deprecated! Use `return_dict` instead!
[115/115 01:39, Epoch 5/5]
```

Step	Training Loss
5	2.847892
10	1.828209
15	1.323348
20	1.127835
25	0.911793
30	0.872179
35	0.836987
40	0.876045
45	0.822846
50	0.791339
55	0.709241
60	0.682739
65	0.685499
70	0.675752
75	0.653497
80	0.571290
85	0.569369
90	0.611582
95	0.546809
100	0.560882
105	0.521498
110	0.547909
115	0.522436

Test again

```
import json

# Counters for correct predictions
correct_intent_predictions = 0
correct_priority_predictions = 0
json_decode_errors = 0

total_samples = len(testing_data)
```



```

print(f"\n>>> Evaluating model on {total_samples} test samples...\n")

for i, sample in enumerate(testing_data):
    input_text = sample["input_text"]
    expected_intent = sample["intent"]
    expected_priority = sample["priority"]

    # Prepare messages for the model to generate the JSON output

    user_prompt_content = f"### Input: {input_text}\n### Output: "
    messages = [
        {"role": "user", "content": user_prompt_content},
    ]

    inputs = tokenizer.apply_chat_template(
        messages,
        tokenize=True,
        add_generation_prompt=True, # This will add <|assistant|> after user message in the template
        return_tensors="pt",
    ).to("cuda")

    # Generate response
    outputs = model.generate(
        input_ids=inputs,
        #max_new_tokens=256,
        use_cache=True,
        temperature=0.1,
        do_sample=True,
        top_p=0.9,
    )

    response = tokenizer.batch_decode(outputs)[0]

    # Extract only the assistant's output (JSON part)
    # The actual output might be something like: <|user|>...<|end|><|assistant|> {JSON}<|end|>
    assistant_prefix = "<|assistant|> "
    end_of_text_suffix = "<|end|>"

    extracted_output = ""
    start_index = response.find(assistant_prefix)
    if start_index != -1:
        start_index += len(assistant_prefix)
        end_index = response.find(end_of_text_suffix, start_index)
        if end_index != -1:
            extracted_output = response[start_index:end_index].strip()

    # Parse the extracted JSON
    model_intent = None
    model_priority = None
    try:
        model_output = json.loads(extracted_output)
        model_intent = model_output.get("intent")
        model_priority = model_output.get("priority")
    except json.JSONDecodeError:
        json_decode_errors += 1
        print(f"Sample {i+1}: Could not decode JSON from model output.")
        print(f"  Extracted output: '{extracted_output}'")
        print(f"  Full model response: '{response}'")

    # Compare and print results
    print(f"--- Sample {i+1} ---")
    print(f"Input: {input_text}")
    print(f"Expected Intent: {expected_intent}, Model Intent: {model_intent}")
    print(f"Expected Priority: {expected_priority}, Model Priority: {model_priority}")

    if model_intent == expected_intent:
        correct_intent_predictions += 1
        print("Intent: Correct")
    else:
        print("Intent: Incorrect")

    if model_priority == expected_priority:
        correct_priority_predictions += 1
        print("Priority: Correct")
    else:
        print("Priority: Incorrect")

```

```

print("\n")

# Print overall accuracy
intent_accuracy = (correct_intent_predictions / total_samples) * 100
priority_accuracy = (correct_priority_predictions / total_samples) * 100

print("\n>>> Evaluation Summary <<<")
print(f"Total Test Samples: {total_samples}")
print(f"Correct Intent Predictions: {correct_intent_predictions}/{total_samples} ({intent_accuracy:.2f}%)")
print(f"Correct Priority Predictions: {correct_priority_predictions}/{total_samples} ({priority_accuracy:.2f}%)")
print(f"JSON Decode Errors: {json_decode_errors}")

>>> Evaluating model on 20 test samples...

/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:71: FutureWarning: The attention mask API under
warnings.warn(DEPRECATION_MESSAGE, FutureWarning)
/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:281: FutureWarning: The attention mask API unde
warnings.warn(DEPRECATION_MESSAGE, FutureWarning)
--- Sample 1 ---
Input: I just need help finding the user manual for this model.
Expected Intent: product_info, Model Intent: product_info
Expected Priority: low, Model Priority: low
Intent: Correct
Priority: Correct

--- Sample 2 ---
Input: The device suddenly stopped responding after the last firmware update. It's unusable.
Expected Intent: technical_support, Model Intent: technical_support
Expected Priority: high, Model Priority: high
Intent: Correct
Priority: Correct

--- Sample 3 ---
Input: I keep receiving suspicious emails asking for my login details. Is this a known scam?
Expected Intent: technical_support, Model Intent: technical_support
Expected Priority: medium, Model Priority: high
Intent: Correct
Priority: Incorrect

--- Sample 4 ---
Input: Are there any promotions running right now for first-time subscribers?
Expected Intent: billing_query, Model Intent: product_info
Expected Priority: medium, Model Priority: low
Intent: Incorrect
Priority: Incorrect

--- Sample 5 ---
Input: Could you elaborate on the difference between silk thread and synthetic fiber options?
Expected Intent: product_info, Model Intent: product_info
Expected Priority: medium, Model Priority: low
Intent: Correct
Priority: Incorrect

--- Sample 6 ---
Input: My account shows an incorrect zip code, please fix that for me.
Expected Intent: billing_query, Model Intent: billing_query
Expected Priority: medium, Model Priority: low
Intent: Correct
Priority: Incorrect

--- Sample 7 ---
Input: I feel like I'm being charged extra just for having two services, is there a bundle?
Expected Intent: billing_query, Model Intent: billing_query

```

Observations

We improved the intent but the priority stayed the same. Questions 3, 4, 5, 9, 19 and 20 were failed in both runs. There was only one question difference between both runs (12 vs 6).

Deciding the priority without explicit urgency clues is very subjective. In fact all instances where the expected priority was "high" were correctly predicted. All failed predictions were only 1 level mistake, ie "medium" when the expected was "low"



Optionally we can generate create a quantized GGUF to run locally

```
model.save_pretrained_gguf("gguf_model", tokenizer, quantization_method="q4_k_m")
```

Start coding or [generate](#) with AI.